

Instruction Set Architecture

- What is an ISA
- RISC Architecture vs CISC
- ARM ISA.
 - Why is important?
 - In almost every smart phone/tablet
 - Fastest growing architecture in the world

Instruction Set Architecture

- Arithmetic operations

add x1,x2,x3; $R[x1] = R[x2] + R[x3]$

sub x1,x2,x3; $R[x1] = R[x2] - R[x3]$

addi x1,x2,5; $R[x1] = R[x2] + 5$

Logic operations: and,or,xor,andi,ori,xori

Instruction Set Architecture

●Shift

sll x5,x5,x7 $x5 = x6 \ll x7$

srl x5,x5,x7 $x5 = x6 \gg x7$

sra x5,x5,x7 $x5 = x6 \ll x7$ BUT Arithmetic

logical shift 0s; arithmetic shifts the sign bit

example: abcd

logical shift: 0abc

arithmetic shift: aabc

srli

srri

slli

Instruction Set Architecture

- Load and store words

`lw x1,4(x2)` $R[x1] = \text{Mem}[R[x2] + 4]$

`sw x1,4(x2)` $\text{Mem}[R[x2] + 4] = R[x1]$

Instruction Set Architecture

● Branch/ jump instructions

Conditional

- Beq x5,x6,label # if($x5 == x6$) goto Pc+label
- Bne
- Blt
- Bge

So:

Branch not Taken: $PC = PC + 4$

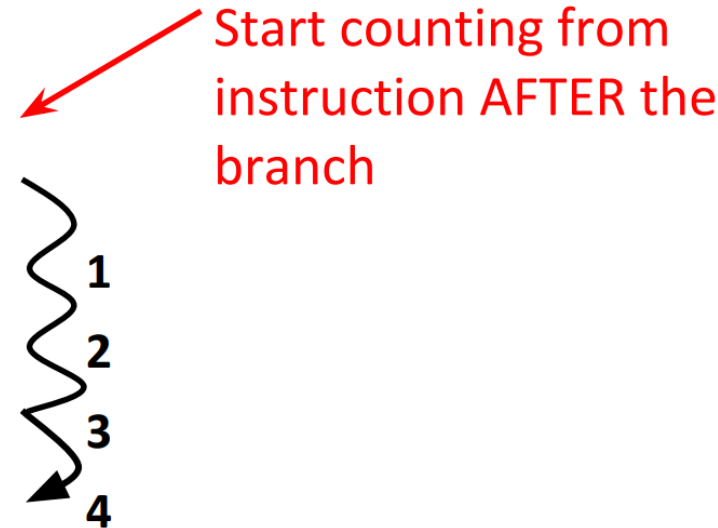
Branch Taken: $PC = PC + (\text{sign extend}(\text{Label} * 4))$

Label is an immediate can be + (forward) or – (backward)

Instruction Set Architecture

- RISC-V Code:

```
Loop: beq x19, x10, End
      add x18, x18, x10
      addi x19, x19, -1
      j Loop
End:  <target instr>
```



- Branch offset = $4 \times 32\text{-bit instructions} = 16\text{ bytes}$
- (Branch with offset of 0, branches to itself)

Instruction Set Architecture

Unconditional Jump

- Jal x1, label # $x1 = PC + 4$ and $PC = PC + \text{Label}$
- Jalr x1, x2(Label) # $x1 = PC + 4$ and $PC = R[x2] + \text{Label}$

X1 used as the return address

Details:

Jal->

$PC = PC(\text{of the jal instruction}) + \text{SignExtend}(\text{Label} \ll 1)$

Jalr->

$PC = \text{SignExtend}(\text{label}) + R[x2]$

Instruction Set Architecture

Far Jumps where far is $>2^{10}$

Beq x10,x0,far

#next

Changed to:

Bne x10,x0,next

J far #far now can be a 20 bits immediate

Next: #next

Even longer: lui x1,far

jalr ra,x1(far last 12 bits)

Big immediate:

Lui x10,0x87645

Addi x10,x10,0x321=> x10=0x87645321

example

sum =0

For(i=0;i<10;i++)

sum+=a[i];

.data

arr: .word 7,1,7,1,8,9,10,11,12,13,14,15

.text

Main:la x8,arr #x8=&arr[0]

add x10,x0,x0 #sum=0

add x11,x0,x0 #i=0

addi x12,x0,10 # x12=10

loop: bge x11,x12,exit

slli x13, x11,2 #mult i by 4

add x14, x13,x8 #x14 is &a[0]+i

lw x15,0(x14) #x15 is &a[i]

add x10,x10,x15 #sum+=a[i]

addi x11,x11,1

j loop

exit:

Leaf Procedure Example

- C code:

```
int leaf_example (int g, h, i, j)
{ int f;
  f = (g + h) - (i + j);
  return f;
}
```

- Arguments g, ..., j in r0, ..., r3
- f in r4

Procedure (Function) Calling

- Steps required
 1. Place input parameters in registers
 2. Transfer control to procedure
 3. Acquire storage for procedure
 4. Perform procedure's operations
 5. Place result in register for caller
 6. Return to place of call

Procedure (Function) Calling

- Registers faster than memory, so use them
- **a0–a7** (**x10–x17**): eight *argument* registers to pass parameters and two return values (**a0–a1**)
- **ra**: one *return address* register to return to the point of origin (**x1**)

Name	Register number	Usage	Preserved on call?
x0	0	The constant value 0	n.a.
x1 (ra)	1	Return address (link register)	yes
x2 (sp)	2	Stack pointer	yes
x3 (gp)	3	Global pointer	yes
x4 (tp)	4	Thread pointer	yes
x5-x7	5-7	Temporaries	no
x8-x9	8-9	Saved	yes
x10-x17	10-17	Arguments/results	no
x18-x27	18-27	Saved	yes
x28-x31	28-31	Temporaries	no

FIGURE 2.14 RISC-V register conventions.

This information is also found in Column 2 of the RISC-V Reference Data Card at the front of this book.

Instruction Support for Functions

```
... sum(a,b);... /* a,b:s0,s1 */  
}  
C int sum(int x, int y) {  
    return x+y;  
}
```

Assembly address (shown in decimal)

1000
1004
1008
1012
1016
...
2000
2004

Instruction Support for Functions cont.

- Single instruction to jump and save return address:

jump and link (**jal**)

- Before:

```
1008 addi ra,zero,1016 #ra=1016  
1012 j sum #goto sum
```

- After:

```
1008 jal sum # ra=1012,goto sum
```

- Why have a **jal**?
 - Make the common case fast: function calls very common
 - Reduce program size
 - Don't have to know where code is in memory with **jal**!

RISC-V

RISC-V Function Call Instructions

- Invoke function: *jump and link* instruction (**j~~a~~l**)
(really should be **l~~a~~j** “*link and jump*”)
 - “link” means form an *address* or *link* that points to calling site to allow function to return to proper address
 - Jumps to address and simultaneously saves the address of the following instruction in register **ra**
 - Assembler shorthand (pseudo instr.):

call FunctionLabel

- Return from function: *jump register* pseudo instruction (**j~~r~~**)
 - Unconditional jump to address specified in register: **j~~r~~ ra**
 - Assembler shorthand (pseudo instr.) :

ret

Example

Example

```
int Leaf
(int g, int h, int i, int j)
{
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

- Parameter variables **g**, **h**, **i**, and **j** in argument registers **a0**, **a1**, **a2**, and **a3**, and **f** in **s0**
- Assume need one temporary register **s1**

Where Are Old Register Values Saved to Restore Them After Function Call?

- Need a place to save old values before call function, restore them when return, and delete
- Ideal is *stack*: last-in-first-out queue (e.g., stack of plates)
 - Push: placing data onto stack
 - Pop: removing data from stack
- Stack in memory, so need register to point to it
- **sp** is the *stack pointer* in RISC-V (**x2**)
- Convention is grow stack down from high to low addresses
 - *Push* decrements **sp**, *Pop* increments **sp**

Assembly Code for Leaf()

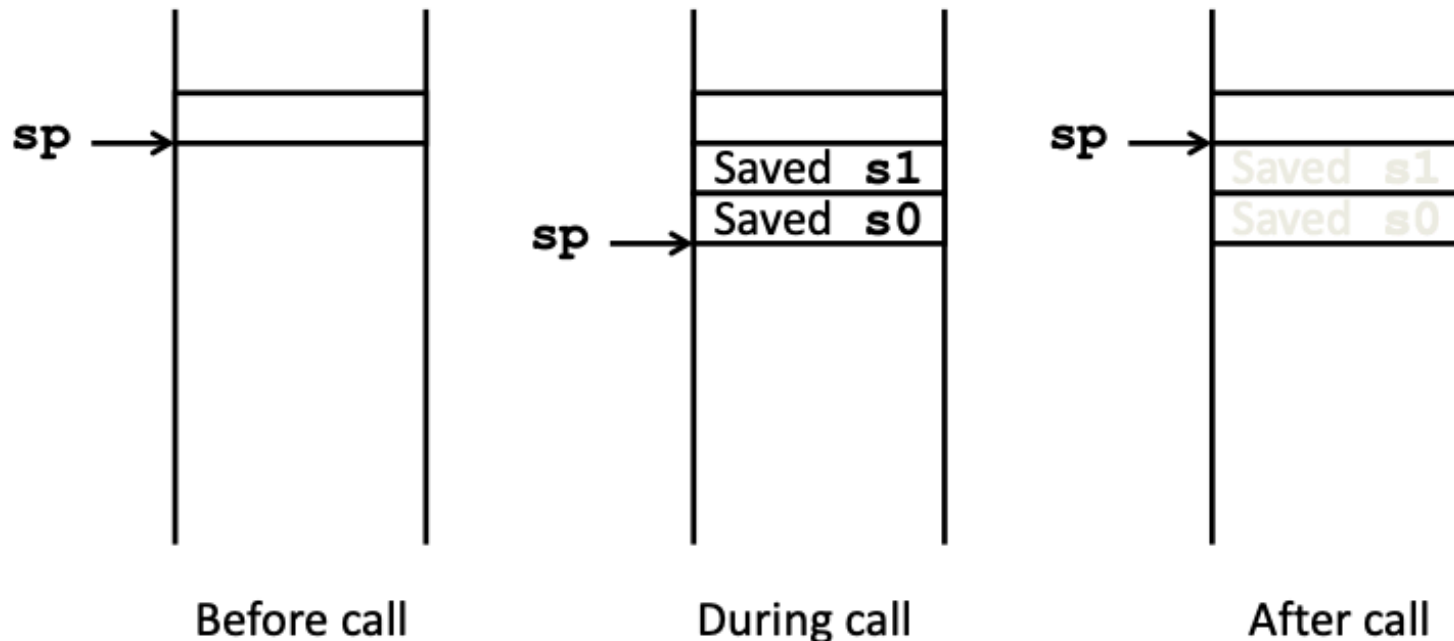
```
Leaf: addi sp,sp,-8 # adjust stack for 2 items
      sw s1, 4(sp)  # save s1 for use afterwards
      sw s0, 0(sp)  # save s0 for use afterwards

      add s0,a0,a1   # f = g + h
      add s1,a2,a3   # s1 = i + j
      sub a0,s0,s1   # return value (g + h) - (i + j)

      lw s0, 0(sp)   # restore register s0 for caller
      lw s1, 4(sp)   # restore register s1 for caller
      addi sp,sp,8    # adjust stack to delete 2 items
      ret            # jump back to calling routine
```

Stack Before, During, After Function

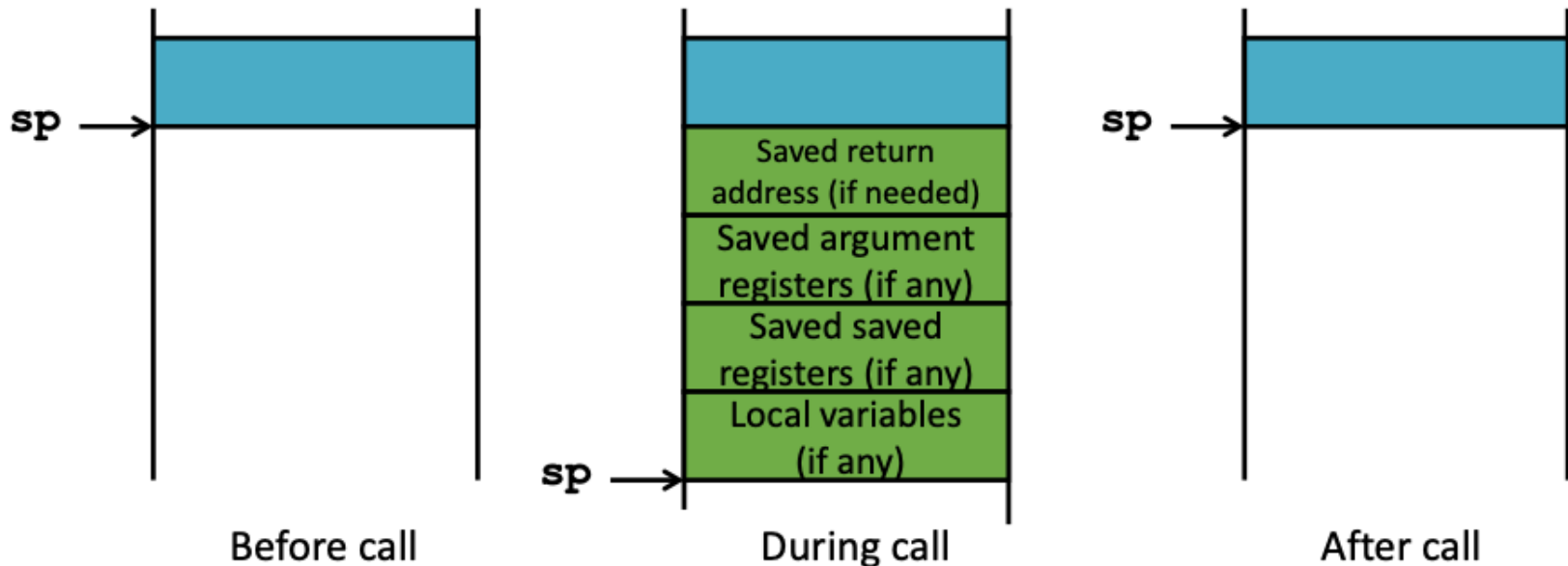
- Need to save old values of **s0** and **s1**



Allocating Space on Stack

- C has two storage classes: automatic and static
 - *Automatic* variables are local to function and discarded when function exits
 - *Static* variables exist across exits from and entries to procedures
- Use stack for automatic (local) variables that don't fit in registers
- *Procedure frame* or *activation record*: segment of stack with saved registers and local variables

Stack Before, During, After Function



Using the Stack (1/2)

- So we have a register **sp** which always points to the last used space in the stack
- To use stack, we decrement this pointer by the amount of space we need and then fill it with info
- So, how do we compile this?

```
int sumSquare(int x, int y) {  
    return mult(x,x)+ y;  
}
```

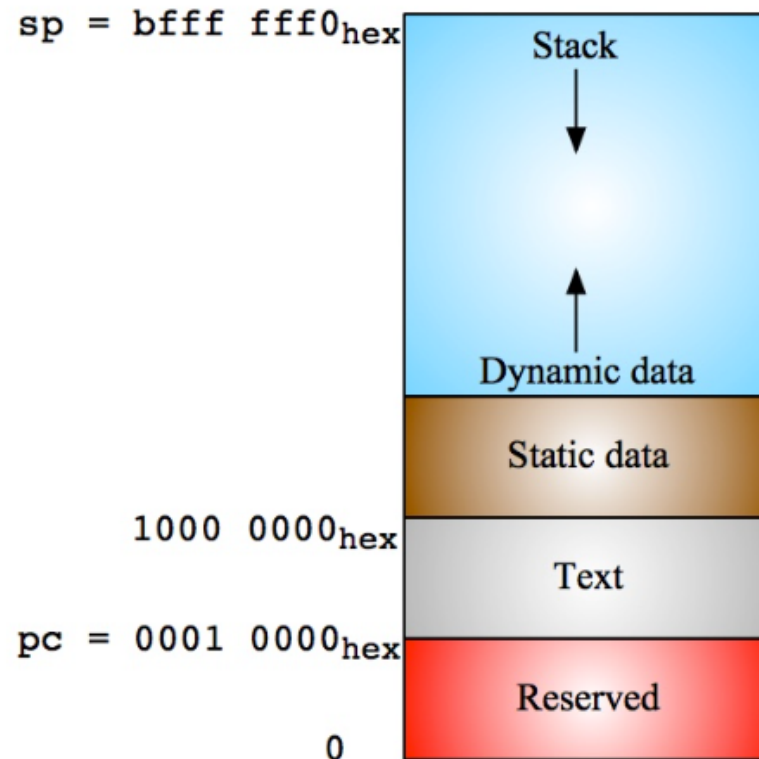
Using the Stack (2/2)

```
int sumSquare(int x, int y) {  
    return mult(x,x)+ y; }
```

sumSquare:

```
    addi sp,sp,-8 # space on stack  
“push”  sw ra, 4(sp)      # save ret addr  
        sw a1, 0(sp)      # save y  
        mv a1,a0          # mult(x,x)  
        jal mult          # call mult  
        lw a1, 0(sp)      # restore y  
        add a0,a0,a1      # mult()+y  
“pop”   lw ra, 4(sp)      # get ret addr  
        addi sp,sp,8      # restore stack  
        jr ra            # ret  
mult:  ...
```

RV32 Memory Allocation



Name (Field size)	Field						Comments
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

FIGURE 2.19 RISC-V instruction formats.

Format	Instruction	Opcode	Funct3	Funct6/7
R-type	add	0110011	000	0000000
	sub	0110011	000	0100000
	sll	0110011	001	0000000
	xor	0110011	100	0000000
	srl	0110011	101	0000000
	sra	0110011	101	0000000
	or	0110011	110	0000000
	and	0110011	111	0000000
	l r.d	0110011	011	0001000
	sc.d	0110011	011	0001100
I-type	lb	0000011	000	n.a.
	lh	0000011	001	n.a.
	iw	0000011	010	n.a.
	id	0000011	011	n.a.
	ibu	0000011	100	n.a.
	ihu	0000011	101	n.a.
	iwu	0000011	110	n.a.
	addi	0010011	000	n.a.
	slli	0010011	001	0000000
	xori	0010011	100	n.a.
	srl i	0010011	101	0000000
	srai	0010011	101	0100000
	ori	0010011	110	n.a.
	andi	0010011	111	n.a.
	jalr	1100111	000	n.a.
S-type	sb	0100011	000	n.a.
	sh	0100011	001	n.a.
	sw	0100011	010	n.a.
	sd	0100011	111	n.a.
SB-type	beq	1100111	000	n.a.
	bne	1100111	001	n.a.
	blt	1100111	100	n.a.
	bge	1100111	101	n.a.
	bltu	1100111	110	n.a.
	bgeu	1100111	111	n.a.
U-type	lui	0110111	n.a.	n.a.
UJ-type	jal	1101111	n.a.	n.a.

FIGURE 2.18 RISC-V instruction encoding.

The *while* loop on page 94 was compiled into this RISC-V assembler code:

```
Loop:slli x10, x22, 3 // Temp reg x10 = i * 8
add x10, x10, x25 // x10 = address of save[i]
ld x9, 0(x10) // Temp reg x9 = save[i]
bne x9, x24, Exit // go to Exit if save[i] != k
addi x22, x22, 1 // i = i + 1
beq x0, x0, Loop // go to Loop
Exit:
```

Answer

If we assume we place the loop starting at location 80000 in memory, what is the RISC-V machine code for this loop?

The assembled instructions and their addresses are:

Address Instruction						
80000	00000000	00011	10110	001	01010	0010011
80004	00000000	11001	01010	000	01010	0110011
80008	00000000	00000	01010	011	01001	0000011
80012	00000000	11000	01001	001	01100	1100011
80016	00000000	00001	10110	000	10110	0010011
80020	11111111	00000	00000	000	01101	1100011

Remember that RISC-V instructions have byte addresses, so addresses of sequential words differ by 4. The `bne` instruction on the fourth line adds 3 words or 12 bytes to the address of the instruction, specifying the branch destination relative to the branch instruction ($12+80012$) and not using the full destination address (80024). The branch instruction on the last line does a similar calculation for a backwards branch ($-20+80020$), corresponding to the label `Loop`.

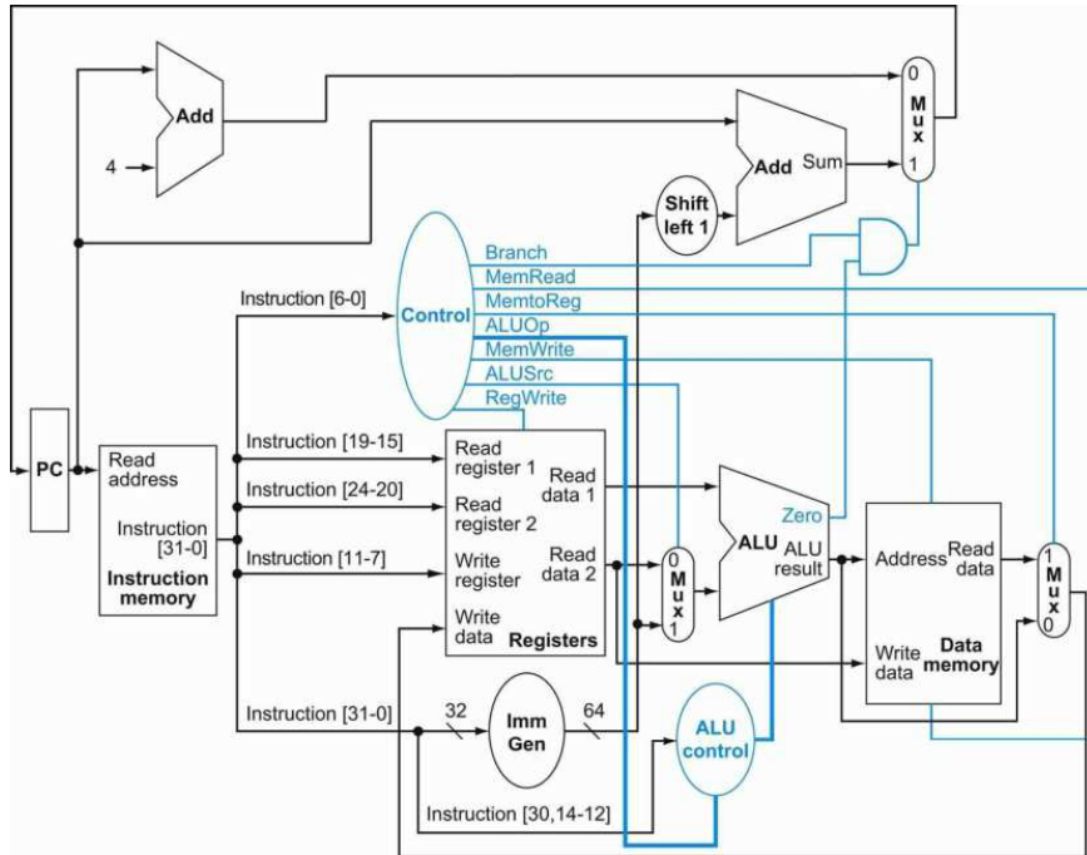


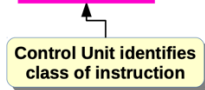
FIGURE 4.17 The simple datapath with the control unit.

Input or output	Signal name	R-format	ld	sd	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

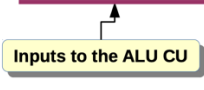
FIGURE 4.22 The control function for the simple single-cycle implementation is completely specified by this truth table.

Truth Table of the ALU CU

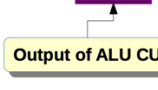
Instrn Opcode	ALU Op	Operation	funct7	funct3	ALU Action	ALU Control Input
LD	00	Load doubleword	XXXXXXX	XXX	add	0010
SD	00	Store doubleword	XXXXXXX	XXX	add	0010
BEQ	01	Branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001



Control Unit identifies
class of instruction



Inputs to the ALU CU



Output of ALU CU